

Senior AI Engineer Take-Home Assignment: The "zooplus Assistant"

Thank you for your interest in the Senior AI Engineer, Agentic Commerce position at zooplus! This assignment is designed to simulate a real-world task and give you a chance to showcase your skills in building and reasoning about production-oriented AI systems.

The task is designed to be completed offline and should take approximately 4-6 hours. We expect and encourage the use of AI coding assistants. We will ask you to present and walk us through your solution in a follow-up interview.

The Challenge

Your task is to build a proof-of-concept for a chatbot API we'll call the "zooplus Assistant." This assistant helps customers find products and answers questions based on a provided product catalog dataset.

zooplus is a multi-shop e-commerce platform operating across different markets (sites). The assistant must be able to search and recommend products within the context of a specific shop.

Functional Requirements

1. API Endpoint

Build an asynchronous backend service in Python using FastAPI. Expose a POST endpoint (e.g., /chat) with the following request contract:

```
{
  "site_id": 3,
  "query": "What's the best dry food for a puppy with a sensitive stomach?"
}
```

- `site_id` (required): Identifies the shop context. The assistant must only return products available in that site.
- `query` (required): The customer's natural language question.

2. Structured Response

The endpoint must return a structured JSON response:

```
{
  "answer": "Based on your needs, I'd recommend...",
  "retrieved_products": {...}
}
```

}

- **answer:** A helpful, conversational response grounded in the retrieved product data.
- **retrieved_products:** A structured output field with article information from the dataset that were used as context to generate the answer.

3. Retrieval-Augmented Generation (RAG)

The assistant's knowledge must come exclusively from the provided product dataset. Implement a RAG pipeline that retrieves relevant products and uses an LLM to synthesize an answer.

4. Guardrails

The assistant must only answer questions related to pet products. Off-topic queries (e.g., "What's the weather today?") should be politely declined.

5. Production-Oriented Design

While this is a PoC, design your solution as if it were the first iteration towards a production service. Your API contract and code structure should reflect scalability considerations, even if the implementation is simplified.

Provided Resources

- A `product_catalog_dataset.json` file containing sample product data.

Submission

Please provide us with a link to a Git repository containing:

Source Code

The complete, runnable source code for your application. This includes:

- The FastAPI application
- Any scripts or code required to prepare data (indexing, embedding, etc.) if applicable
- Clear setup instructions so we can run it locally

README.md

This is a critical part of your submission. Please structure it with:

1. **High-Level Design:** A brief overview of your solution's architecture. A simple diagram is welcome.

2. **Setup and Execution:** Clear instructions to set up the environment and run the project locally.
3. **Decisions and Trade-offs:** Explain the key decisions you made in designing your agent and retrieval pipeline. What trade-offs did you consciously accept for the sake of this PoC, and what would you change or recommend for a production deployment?
4. **Future Roadmap:** If this PoC were approved, what would be the next 3-5 steps to move it towards production? (Think about testing, observability, evaluation, scalability, additional features, etc.)

Evaluation Criteria

We value:

- **Engineering rigor and justified decisions** over complexity or feature count.
- **Understanding of agentic flows and RAG pipelines** — how you reason about retrieval, context, and generation.
- **Data awareness** — how well your solution leverages the structure of the provided dataset.
- **Trade-off transparency** — simplified implementations are perfectly acceptable when clearly explained and justified.